

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	Sree B	Reg. No.:	192524023
Section / Batch:	AI and DS / 2025	Date:	21/08/2026

Code of Conduct

I Sree B(192524023) _____ (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Sree B _____

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20
Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS Online Lab

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=514

Ask Gemini

School

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

SREE B

192524023RA

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Q4

Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Q2 Graph - Topological Sort

20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

a. Detect cycle before sorting

b. Use DFS approach

Your Answer

TopologicalSort_DFS(graph, V):

state[v] = 0 for all vertices v // 0 = unvisited, 1 = visiting, 2 = visited

stack = empty

for each vertex u in 0..V-1:

if state[u] == 0:

Save Answer

Next →

SIMATS Online Lab

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=514

Ask Gemini

School

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

SREE B

192524023RA

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Q4

Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

a. All zones must be connected

b. Total installation cost must be minimized

c. No redundant connections (no cycles) allowed

d. The network must remain fully connected

Your Answer

Algorithm Kruskal(G)

Input: A connected, undirected graph $G = (V, E)$ where V is the set of zones and E is the set of possible cable connections with weights w .

Output: A Minimum Spanning Tree (MST) that connects all zones with minimum total cost.

1. Create an empty set MST to store the selected edges.

Save Answer

Next →

SIMATS Online Lab

Ask Gemini

←

→

↻

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=514

☆

📁

S School

⋮

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

SREE B

192524023RA

▼

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Q4

Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Q3 Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

a. Use MST for infrastructure

b. Use shortest path for routing

Your Answer

ALGORITHM SmartTransportation_MST_Dijkstra(G, Wroad, Source)

G = Graph of locations (nodes), edges represent roads between locations

Wroad(u,v) = road distance/time cost for edge (u,v)

Source = starting location for routing

Output:

- MST edges for infrastructure

Save Answer

Next →

SIMATS Online Lab

Ask Gemini

←

→

↻

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=514

☆

📁

S School

⋮

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

SREE B

192524023RA

▼

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Q4

Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST.

Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

ALGORITHM SecondBestMST(Graph G):

Input: G = (V, E) with weights w(u,v)

Output: secondBestWeight (weight of second best MST)

1. mstEdges, mstWeight = KruskalMST(G)

// mstEdges = set of edges included in one MST

Save Answer

SIMATS Online Lab

Ask Gemini

Not secure

simatslab.saveetha.com/practice_app/classactivity/attempt?assignment_id=514

☆

S School

SIMATS

ENGINEERING

SIMATS Online Programming Lab

DS PROGRAMMING

SREE B

192524023RA

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm

20 marks

Q2

Graph - Topological Sort

20 marks

Q3

Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's

20 marks

Q4

Shortest Path Algorithm -
Dijkstra's Algorithm

20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

20 marks

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

```
FUNCTION DijkstraWithVoucher(Graph, StartNode, EndNode):  
  // dist0[u] is the min distance to u without using the voucher  
  // dist1[u] is the min distance to u having used the voucher once  
  dist0 = array of size |Graph| initialized to INFINITY  
  dist1 = array of size |Graph| initialized to INFINITY
```

Save Answer

Next →